the same value. The tests can assign a value to the private @time_class variable by using either class_eval or instance_eval. Either of the two methods will do here, because they both change self:

```
class FakeTime
  def self.now; 'Mon Apr 06 12:15:50'; end
end

require 'test/unit'

class TestLoan < Test::Unit::TestCase
  def test_conversion_to_string
    Loan.instance_eval { @time_class = FakeTime }
    loan = Loan.new('War and Peace')
    assert_equal 'WAR AND PEACE loaned on Mon Apr 06 12:15:50', loan.to_s
  end
end
```

Bill is quite proud of his own coding prowess. He says, "I think we deserve a break—after I give you a quiz."

Quiz: Class Taboo

Where you write an entire program without ever using a certain popular keyword.

Did you ever play *Taboo*?[1] The rules are simple: you're given a secret word and a list of words that you cannot use. (They are "taboo.") You must help a teammate guess the secret word. You can give your teammate as many suggestions as you want, but you must never say a taboo word. If you do that, you lose immediately.

Your challenge: play *Taboo* with Ruby code. You have only one taboo word, the class keyword. Your "secret word" is actually a Ruby class:

```
class MyClass < Array
  def my_method
    'Hello!'
  end
end
```

You have to write a piece of code that has exactly the same effect as the previous one, without ever using the class keyword. Are you up to the challenge? (Just one hint: look at the documentation for Class.new.)

1. http://en.wikipedia.org/wiki/Taboo_(game).